UNIDAD 6

Trabajo práctico 06 - Refactorización modular de reservas

1. Información y entrega

- **Modalidad:** individual.
- **Dominio obligatorio:** reservas de salas de estudio.
- **Punto de partida:** una copia funcional del TP 05 de la misma persona.
- **Entrega:** URL de un repositorio individual de GitHub.
- **Vencimiento:** lunes siguiente a las 19:59, antes de la clase de las 20:00.
- **Ejecución mínima:** `npm install` , `npm start` , `npm run check` .

Este TP evalúa la **organización** de una aplicación existente, no la incorporación de una nueva funcionalidad. La entrega anterior debe quedar intacta. Nombrar la copia `tp-06-salas-modular` ; no trasladar `node_modules` ni `.git` del proyecto anterior. Si el TP 05 no quedó completo, primero se debe recuperar una línea de base ejecutable con el contrato mínimo de unidad 5 y documentar qué faltaba, antes de atribuir los cambios a la refactorización.

2. Consigna

Reorganizar la aplicación de reservas de salas en módulos de arranque, configuración, aplicación, rutas, controladores, servicio y middleware. Conservar las mismas vistas EJS, el layout, el CSS, los datos iniciales, Morgan, identificador de solicitud, medición hasta `finish` , validación de POST, alcances y respuestas HTTP. No agregar base de datos ni persistencia.

Una persona que solo navega el sitio o usa sus rutas debe observar el mismo comportamiento antes y después, excepto que el puerto y el formato de registro ahora pueden configurarse desde el entorno.

3. Línea de base obligatoria

Antes de mover código, completar una tabla propia con resultado observado para:

```
GET  /
GET  /estado
GET  /reservas
GET  /reservas/nueva
GET  /reservas/:id
POST /reservas   válido e inválido
GET  /css/estilos.css
GET  /url-inexistente
```

Registrar estado HTTP, contenido o vista, cantidad de reservas y una evidencia breve. El POST válido produce 302 y luego un GET 200; el inválido responde 400 sin crear. El detalle y la URL inexistente responden 404 cuando corresponde. Si la base tenía otra conducta, indicar la reparación realizada antes de refactorizar.

4. Estructura esperada

```
tp-06-salas-modular/
|-- public/                   # conservar CSS y otros recursos
|-- views/                    # conservar layout y vistas
|-- src/
|   |-- index.js              # crea dependencias y arranca
|   |-- app.js                # configura Express y monta routers
|   |-- configuracion.js      # procesa process.env
|   |-- rutas/reservas.js
|   |-- controladores/reservas.js
|   |-- servicios/reservas.js
|   `-- middleware/
|       |-- solicitudes.js    # ID y tiempo
|       `-- reservas.js       # sección y validación
|-- .env.example
|-- .gitignore
|-- eslint.config.js
|-- package.json
|-- package-lock.json
`-- README.md
```

Se permiten nombres equivalentes en español, siempre que el README incluya un mapa inequívoco. No se exige crear archivos vacíos solo para imitar el árbol. La vista `no-encontrado.ejs` y las vistas de reservas no deben desaparecer. Mantener CommonJS.

5. Responsabilidades y dependencias

Módulo	Debe hacer	No debe hacer
`index.js`	Crear la semilla/servicio, leer configuración, crear app y escuchar	Declarar rutas o validar formularios
`configuracion.js`	Convertir y validar variables de entorno	Crear Express o imprimir secretos
`app.js`	Configurar vistas y pipeline; montar rutas; 404 final	Ejecutar `listen` , manipular directamente el arreglo
`rutas/reservas.js`	Relacionar método/camino con middleware y controlador	Duplicar `/reservas` en rutas internas
`controladores/reservas.js`	Leer parámetros, renderizar, enviar JSON, 404 y redirigir	Declarar `app.listen` o almacenar el arreglo
`servicios/reservas.js`	Listar, buscar, contar y crear reservas en memoria	Usar `req` , `res` , EJS o códigos HTTP
`middleware/*`	Conservar ID, medición, sección y validación	Perder `next()` o continuar después de responder 400

El servicio debe ser **una sola instancia** compartida por listado, detalle, `/estado` y creación. Puede construirse como `crearServicioReservas(datosIniciales)` . El controlador puede construirse como `crearControladorReservas(servicio)` . Si `/estado` consulta un arreglo diferente al que modifica el POST, la refactorización es incorrecta.

6. Contrato funcional que se conserva

```
GET  /                  -> 200, inicio
GET  /estado            -> 200, JSON con cantidad e ID BIB-
GET  /reservas          -> 200, listado
GET  /reservas/nueva    -> 200, formulario
GET  /reservas/:id      -> 200 o 404 HTML
POST /reservas          -> 302 válido; 400 inválido
GET  /css/estilos.css   -> 200
otra URL               -> 404 HTML
```

`/nueva` debe registrarse antes de `/:id` . La validación conserva campos, reglas, mensajes y valores previos del TP 05: estudiante, email con `@` , sala permitida, fecha, turno permitido y personas enteras entre 1 y 6. El parser `express.urlencoded` sigue antes del validador. El camino inválido responde y **no** llama a `next()` ; el válido prepara `req.reservaValidada` y continúa hacia el controlador. Los nuevos registros desaparecen después de reiniciar.

El pipeline debe seguir siendo reconocible:

```
Morgan -> identificarSolicitud -> medirDuracion -> layouts -> static
       -> urlencoded -> json -> rutas de aplicación -> router /reservas -> 404
```

Se puede mantener el orden alternativo de `static` y Morgan si ya estaba documentado en el TP 05. Explicar qué solicitudes quedan registradas.

7. Configuración externa

`configuracion.js` debe leer `PORT` y `NODE_ENV` desde `process.env` al inicio, convertir `PORT` a número y rechazar valores no enteros o fuera de 1-65535 con un mensaje claro. Si `PORT` no existe, usar 3000. Utilizar Morgan `dev` en desarrollo y `combined` cuando `NODE_ENV=production` ; documentar que esto **solo** diferencia la configuración del registro.

Versionar `.env.example` con valores **no secretos**:

```
PORT=3000
NODE_ENV=development
```

Agregar `.env` y `node_modules/` a `.gitignore` . `npm start` debe funcionar sin `.env` por los valores predeterminados. Puede agregarse `start:local` con `node --env-file=.env src/index.js` , documentando que requiere un archivo `.env` local. No publicar `.env` , claves ni credenciales. Probar un puerto válido alternativo y un puerto inválido.

8. Calidad de código

Instalar como dependencias de desarrollo:

```
npm install --save-dev prettier eslint @eslint/js
```

Configurar ESLint para los archivos CommonJS de `src` , según el ejemplo de la clase 1. Incluir scripts:

```
{
  "scripts": {
    "start": "node src/index.js",
    "format": "prettier --write src",
    "format:check": "prettier --check src",
    "lint": "eslint src",
    "check": "npm run format:check && npm run lint"
  }
}
```

El bloque muestra los scripts mínimos, no reemplaza el resto de `package.json` . Ejecutar `npm run format` antes de `npm run check` . Entregar `package-lock.json` actualizado. El verificador estático no sustituye la matriz HTTP.

9. Matriz de pruebas manuales

Caso	Esperado	Evidencia a registrar
Inicio y CSS	200	Navegación, recurso cargado, ID visible
Estado inicial	200	JSON con cantidad inicial e ID
Listado y detalle existente	200	Mismos datos de TP 05
Detalle inexistente	404	Vista HTML; sin caída del servidor
Formulario	200	Campos y valores de selección
POST vacío	400	Error; sin alta
Sala no permitida	400	Sin alta
Email sin `@`	400	Sin alta
Personas 0, 7 o fraccionarias	400	Sin alta
POST válido	302 y luego 200	Nueva reserva en listado y `/estado`
URL inexistente	404	Vista HTML final
Reinicio	200	Regreso a la semilla inicial
Puerto alternativo	200	Responde en puerto configurado
Puerto inválido	No inicia	Mensaje explícito

Comprobar que Morgan y la medición propia siguen mostrando 200, 302, 400 y 404. Anotar el resultado **antes y después** para al menos inicio, listado, formulario, POST inválido, POST válido y 404. Una captura o texto breve basta; no se exigen pruebas automatizadas.

10. README de la entrega

Incluir instrucciones reproducibles:

```
# Trabajo práctico 06

## Proyecto de partida y cambios
## Instalación y ejecución
## Configuración del entorno
## Mapa de módulos y dependencias
## Pipeline y contrato de rutas
## Matriz antes/después
## Formato y análisis estático
## Persistencia temporal y límites
```

Explicar con palabras propias por qué el servicio no usa `res` , qué hace el controlador, por qué el router declara caminos relativos, dónde vive el único arreglo de reservas y qué ocurrirá con él al reiniciar. Indicar los comandos ejecutados y sus resultados. No incluir contenido de `.env` local.

11. Criterios de evaluación

Criterio	Puntaje
Copia, línea de base y paridad funcional	2
Separación de arranque y configuración	1
Aplicación y orden del pipeline conservados	1
Router, controlador y servicio con límites claros	2,5
Middleware, validación y estados HTTP intactos	1
Variables de entorno y `.gitignore`	1
Formato, ESLint, scripts y README explicativo	1,5
Total	**10**

12. Fuera de alcance y comprobación final

No se evalúan MongoDB, Mongoose, escritura de archivos, APIs nuevas, autenticación, autorización, manejo centralizado de errores ni pruebas automáticas. Incorporarlos no compensa una ruta rota o un módulo con responsabilidades mezcladas.

Antes de entregar: ejecutar `npm install` , `npm run format` , `npm run check` y `npm start` ; repetir la matriz; confirmar que `package-lock.json` está incluido y `.env` / `node_modules` no están en Git; abrir la URL del repositorio desde una sesión no autenticada si es público o confirmar el acceso solicitado por la institución.